

Homework 5: Statistical Phylogenetics

Bernhard Buss Dario Franzen Aurimas Greicius
Timothy Lauren Nijhuis Carlo Rossetto Yuliyang Slavchev
Görkem Kadir Solun

Problem 12: Intuition on time-reversible Markov chains

Part 1: Transition probabilities of the time-reversed process

Let $\{X(t), 0 \leq t\}$ be a homogeneous continuous-time Markov chain starting from the stationary distribution, meaning $P(X(0) = k) = \pi_k$ for all states. Because the chain starts in the stationary distribution, the marginal probability of being in any state k at *any* time t is π_k .

We define the time-reversed process as $\hat{X}(t) := X(\tau - t)$. We want to find its transition probabilities:

$$\hat{T}_{kl}(s, t) = Pr(\hat{X}(s + t) = l \mid \hat{X}(s) = k)$$

Substitute the definition of the reversed process:

$$\hat{T}_{kl}(s, t) = Pr(X(\tau - (s + t)) = l \mid X(\tau - s) = k)$$

By the definition of conditional probability ($P(A|B) = \frac{P(A,B)}{P(B)}$):

$$\hat{T}_{kl}(s, t) = \frac{Pr(X(\tau - s - t) = l, X(\tau - s) = k)}{Pr(X(\tau - s) = k)}$$

Let $u = \tau - s - t$. Notice that u is the earlier time in the forward process, and $u + t = \tau - s$ is the later time. We rewrite the probabilities in chronological order for the forward process:

$$\hat{T}_{kl}(s, t) = \frac{Pr(X(u) = l, X(u + t) = k)}{Pr(X(u + t) = k)}$$

Using the product rule ($P(A, B) = P(A)P(B|A)$), we can expand the numerator:

$$\hat{T}_{kl}(s, t) = \frac{Pr(X(u) = l) \cdot Pr(X(u + t) = k \mid X(u) = l)}{Pr(X(u + t) = k)}$$

Because the process is stationary, $Pr(X(u) = l) = \pi_l$ and $Pr(X(u+t) = k) = \pi_k$. Furthermore, the forward conditional probability is just the standard transition matrix $T_{lk}(t)$:

$$\hat{T}_{kl}(s, t) = \frac{\pi_l T_{lk}(t)}{\pi_k}$$

This result depends only on the duration t and not on the specific starting time s . Therefore, the time-reversed process is also homogeneous, and we can write:

$$\hat{T}_{kl}(t) = \frac{\pi_l T_{lk}(t)}{\pi_k} \quad \square$$

Part 2: Equivalence for time-reversible chains

Assume the original Markov chain is time-reversible. By definition, this means:

$$\pi_k T_{kl}(t) = \pi_l T_{lk}(t)$$

From Part 1, we established that the transition probability of the reversed process is:

$$\hat{T}_{kl}(t) = \frac{\pi_l T_{lk}(t)}{\pi_k}$$

Rearranging the reversibility condition gives us $\frac{\pi_l T_{lk}(t)}{\pi_k} = T_{kl}(t)$. Substituting this into our equation for the reversed process yields:

$$\hat{T}_{kl}(t) = T_{kl}(t)$$

This proves that a time-reversible Markov chain looks statistically identical whether viewed forwards or backwards in time. $\square$

Problem 13: Phylogenetic trees as Bayesian networks

Part 1: Joint probability and naive marginalization

Based on the directed edges of the Bayesian network T , the joint probability factorizes as the product of the probabilities of each node given its parent:

$$P(Y, Z | T) = P(Z_1) \cdot P(Y_4 | Z_1) \cdot P(Z_2 | Z_1) \cdot P(Y_3 | Z_2) \cdot P(Z_3 | Z_2) \cdot P(Y_2 | Z_3) \cdot P(Y_1 | Z_3)$$

To find the marginal probability $P(Y | T)$, we must naively sum over all possible configurations of the hidden variables $Z = (Z_1, Z_2, Z_3)$:

$$P(Y | T) = \sum_{Z_1} \sum_{Z_2} \sum_{Z_3} P(Z_1) P(Y_4 | Z_1) P(Z_2 | Z_1) P(Y_3 | Z_2) P(Z_3 | Z_2) P(Y_2 | Z_3) P(Y_1 | Z_3)$$

Number of summation terms (naive): There are 3 hidden variables (Z_1, Z_2, Z_3), and each can take one of 4 nucleotide states (A, C, G, T). To evaluate this naively, we must compute a term for every combination, requiring $4^3 = 64$ summation terms.

Part 2: Felsenstein's Peeling Algorithm

We can use the distributive law to push the sums inward, evaluating them from the leaves (bottom) to the root (top).

$$P(Y | T) = \sum_{Z_1} P(Z_1) P(Y_4 | Z_1) \left[\sum_{Z_2} P(Z_2 | Z_1) P(Y_3 | Z_2) \left[\sum_{Z_3} P(Z_3 | Z_2) P(Y_2 | Z_3) P(Y_1 | Z_3) \right] \right]$$

Number of summation terms (peeling):

Innermost sum (over Z_3): Computed for each of the 4 states of parent Z_2 . Each sum iterates over the 4 states of Z_3 . This yields $4 \times 4 = 16$ terms.

Middle sum (over Z_2): Computed for each of the 4 states of parent Z_1 . Each sum iterates over the 4 states of Z_2 . This yields $4 \times 4 = 16$ terms.

Outermost sum (over Z_1): Iterates over the 4 states of the root Z_1 . This yields 4 terms.

At most, this algorithm requires $16 + 16 + 4 = 36$ summation terms.

Problem 14: Learning phylogenetic trees from sequence alignment data

```
library(ape)
library(phangorn)
alignment <- read.dna("ParisRT.txt")
```

Step 2: Initial Tree Topology (Neighbour Joining)

```
# Calculate distances and build NJ trees for the three models
dist_k80 <- dist.dna(alignment, model = "K80")
tree_k80 <- NJ(dist_k80)

dist_tn93 <- dist.dna(alignment, model = "TN93")
```

```

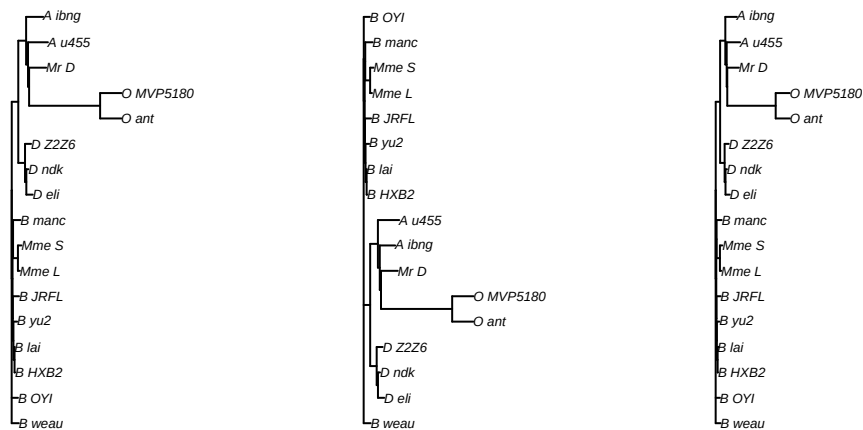
tree_tn93 <- NJ(dist_tn93)

dist_jc69 <- dist.dna(alignment, model = "JC69")
tree_jc69 <- NJ(dist_jc69)

par(mfrow=c(1,3))
plot(tree_k80, main="NJ: Kimura (1980) (K80)")
plot(tree_tn93, main="NJ: Tamura and Nei 1993 (TN93)")
plot(tree_jc69, main="NJ: Jukes and Cantor 1969 (JC69)")

```

NJ: Kimura (1980) (K80) NJ: Tamura and Nei 1993 (TN93) NJ: Jukes and Cantor 1969 (JC69)



Step 3: Fitting the Kimura Model (K80)

```

# Convert data to phyDat format required by phangorn
phyDat_align <- phyDat(alignment)

# Fit the base pml model using K80
fit_k80 <- pml(tree_k80, phyDat_align, model = "K80")

# Print fit, incl. the loglik:
print(fit_k80)

```

```
model: K80
loglikelihood: -3003.487
unconstrained loglikelihood: -2098.897
```

Rate matrix:

```
  a c g t
a 0 1 1 1
c 1 0 1 1
g 1 1 0 1
t 1 1 1 0
```

Base frequencies:

```
  a    c    g    t
0.25 0.25 0.25 0.25
```

Step 4: Optimizing the Substitution Rate Matrix (Q)

```
# Optimise ONLY the parameters of the substitution model (optQ)
fit_k80_optQ <- optim.pml(fit_k80,
                          optQ = TRUE,
                          optNni = FALSE,
                          optBf = FALSE,
                          optEdge = FALSE,
                          optInv = FALSE,
                          optGamma = FALSE,
                          optRate = FALSE,
                          optRooted = FALSE)
```

```
optimize rate matrix: -3003.487 --> -2873.259
optimize rate matrix: -2873.259 --> -2873.259
```

```
# Output optimized rate matrix
cat("Optimized K80 Rate Matrix (Transitions/Transversions):",
    print(fit_k80_optQ), "\n")
```

```
model: K80
loglikelihood: -2873.259
unconstrained loglikelihood: -2098.897
```

Rate matrix:

	a	c	g	t
a	0.000000	2.5171610	7.1159677	1.238374
c	2.517161	0.0000000	0.6187311	7.212688
g	7.115968	0.6187311	0.0000000	1.000000
t	1.238374	7.2126878	1.0000000	0.000000

Base frequencies:

a	c	g	t
0.25	0.25	0.25	0.25

Optimized K80 Rate Matrix (Transitions/Transversions):

Step 5: Full Optimization

```
# Optimise topology (optNni), branch lengths (optEdge), and rates (optQ)
fit_full <- optim.pml(fit_k80,
                      optQ = TRUE,
                      optNni = TRUE,
                      optBf = FALSE,
                      optEdge = TRUE,
                      optInv = FALSE,
                      optGamma = FALSE,
                      optRate = FALSE,
                      optRooted = FALSE)
```

```
optimize edge weights: -3003.487 --> -2992.981
optimize rate matrix: -2992.981 --> -2863.264
optimize edge weights: -2863.264 --> -2862.477
optimize topology: -2862.477 --> -2849.886 NNI moves: 5
optimize rate matrix: -2849.886 --> -2849.791
optimize edge weights: -2849.791 --> -2849.789
optimize topology: -2849.789 --> -2849.789 NNI moves: 0
optimize rate matrix: -2849.789 --> -2849.789
optimize edge weights: -2849.789 --> -2849.789
optimize rate matrix: -2849.789 --> -2849.789
optimize edge weights: -2849.789 --> -2849.789
```

```
print(fit_full)
```

```
model: K80
loglikelihood: -2849.789
```

unconstrained loglikelihood: -2098.897

Rate matrix:

	a	c	g	t
a	0.000000	2.423373	7.123735	1.148312
c	2.423373	0.000000	0.622454	7.541759
g	7.123735	0.622454	0.000000	1.000000
t	1.148312	7.541759	1.000000	0.000000

Base frequencies:

a	c	g	t
0.25	0.25	0.25	0.25

Step 6 & 7: Bootstrapping and Forensic Analysis

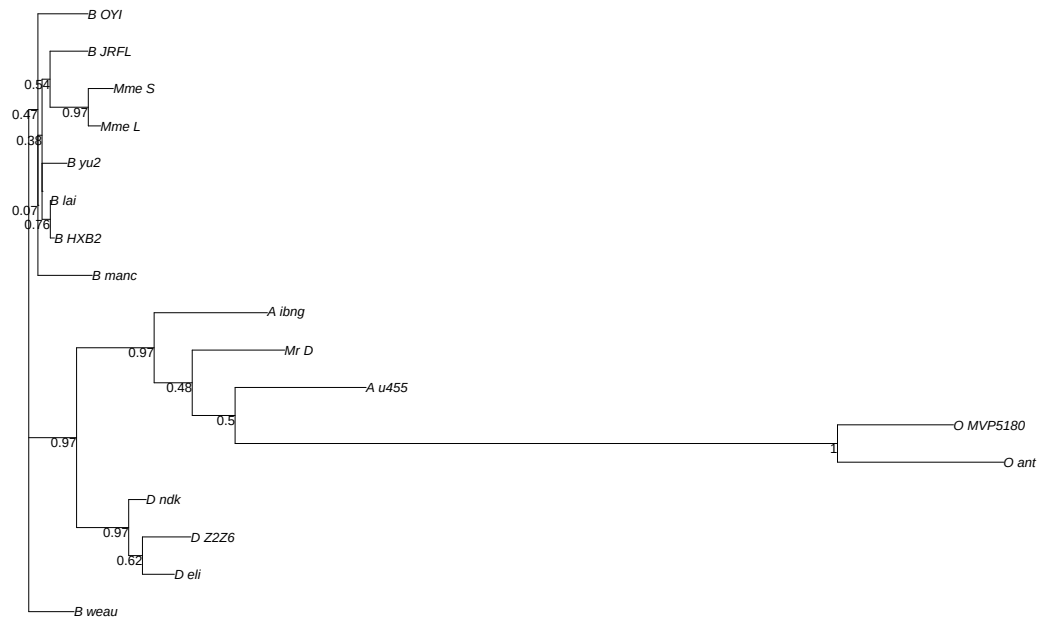
Question: What exactly is being resampled?

Answer: The columns (sites) of the multiple sequence alignment are resampled with replacement.

```
bs <- bootstrap.pml(fit_full, optNni = TRUE, bs = 100)
```

```
plotBS(fit_full$tree,  
      bs,  
      type = "phylogram",  
      main = "Optimised K80 Tree with Bootstrap"  
      )
```

Optimised K80 Tree with Bootstrap



Forensic Conclusion: The nurse closest to the patient “Mme_L” is “Mme S”, thus she is more likely to have infected the patient.